

Présentation

POPOLITICS

Comprendre ce que décident vos élus.
En toute clarté

POPolitics

Pourquoi Popolitics?

Le système politique manque de lisibilité, de transparence et de preuves concrètes d'intégrité.

26%

des Français font
confiance à la politique

CEVIPOF 2025

24%

font confiance à l'AN
ou au Sénat

CEVIPOF 2025

87%

estiment que les élus
sont corrompus

Transparency France

Notre objectif

POPolitics est une application citoyenne, augmentée par la data et l'IA, qui facilite l'accès à la réalité des décisions politiques.

Permettre à n'importe qui de comprendre . . .

- Comment votent les députés
- Comment évoluent leurs positions
- Quels sujets dominent le débat
- Comment se structurent les forces politiques

POPolitics → Un des points d'entrée incontournables pour réintroduire confiance et lisibilité.

POP Politics

éclaire, simplifie et participe à rendre les citoyens actifs dans la vie politique.



Transparence



Lisibilité



Confiance

Expérience Utilisateur

Portail Collectif

Le cœur de la plateforme — analyser les votes, tendances et débats



Suivi des votes

Mise à jour quotidienne, filtres avancés (thème, groupe, élu), codes couleurs Pour/Contre/Abstention.



Grandes tendances

Détection de sujets émergents, indice d'opposition, cartographie des alliances, heatmaps.



Synthèses automatiques

Résumé en 5 points clés par débat, mots forts, ton des interventions via NLP.



Cohérence des élus

Indice de stabilité, alignement au groupe, votes atypiques et profils émergents.



Visualisations interactives

Graphes de réseau, clusters idéologiques (t-SNE), chronologies législatives.

Portail Collectif



Vue simplifiée des élus suivis

« Qu'a fait ce représentant ? »

- Timeline : interventions, amendements, rendez-vous publics
- Profil express : bio, rôle, commissions
- Mini-carte photo + tendance des votes



Fiches éclair express

« Comprendre l'essentiel en < 1 min »

- Résumé : objectif, mécanisme, impacts
- Qui est pour / contre ? Pourquoi ?
- Carte d'influence : lobbies, organisations, acteurs

Page Personnelle

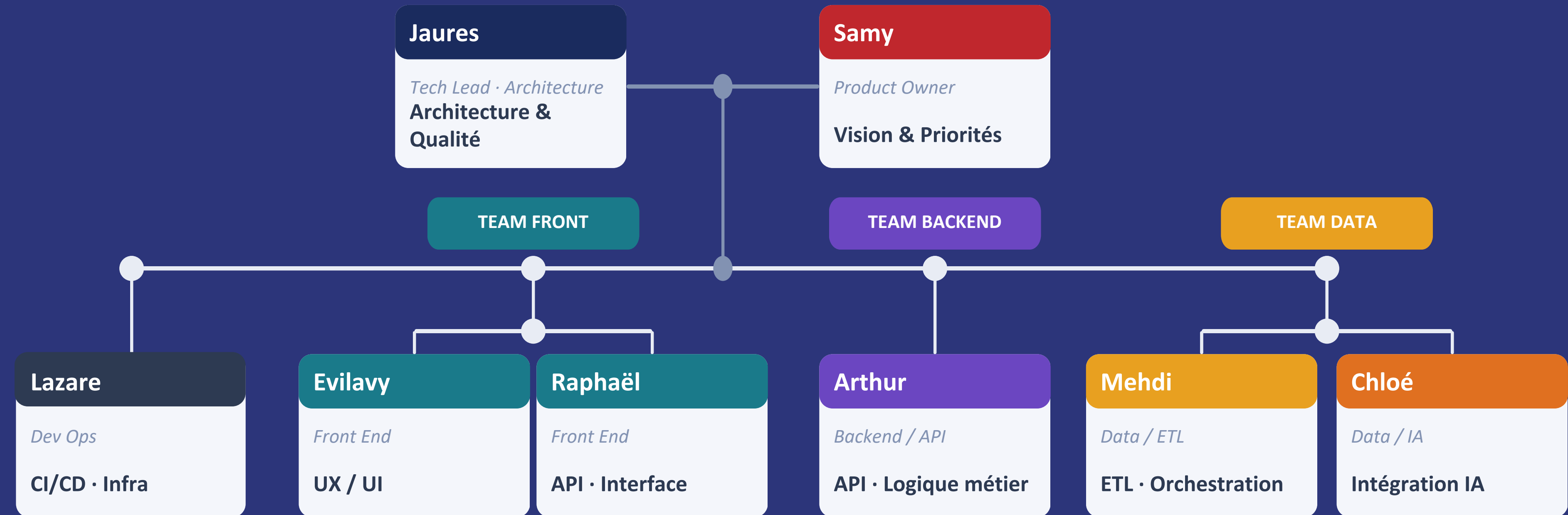
Personalisation de la page selon des favoris utilisateur et filtres de préférence

- Priorisation : importance + urgence + intérêt utilisateur
- Apprentissage dynamique des centres d'intérêt
- Alertes vote majeur, changement de position, amendement



Organisation

Organigramme



Organisation / GitHub



Teams

- Frontend
- Backend
- CI/CD
- DATA/IA



Popolitics

2 followers

France

popolitics@protonmail.com

- Documentation
- Infra
- Data
- Frontend
- ia-service
- data-service
- auth-service
- api



CI.CD



GitHub Actions

Planification

Fonctionnement – Gouvernance Horizontale

Méthode Scrum (adapté)

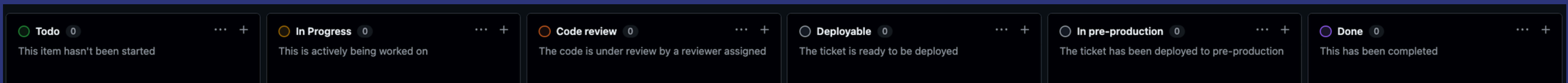


Board GitHub Projects

Visible par tous · Mis à jour en continu

Chaque carte a un responsable + statut

Permet transparence sur l'avancée des tâches



Cérémonies Scrum

Sprint Planning

J1 de chaque sprint — objectif & backlog

Daily Standup

Asynchrone — avancement & blocages

Affinage du Backlog

En milieu de sprint pour découper et estimer

Sprint Review & Rétro Sprint

Dernier jour — démo de l'incrément
Après Review — amélioration continue

Story Points

XS-2 Quelques heures

S-3 Demi-journée

M-5 1–2 jours

L-8 À découper

Priorisation fonctionnelle

MUST

- Auth (compte, login, rôles)
- ETL AN + Sénat + UE (Bronze+Silver)
- Backend Django (Data/Auth Service)
- Portail collectif MVP (votes + filtres)

SHOULD

- Couche Gold + Cross-match
- Premier modèle IA (chatbot)
- Page personnelle v1 (suivis + notifs)
- Filtres avancés + indicateurs élus

COULD

- Modèles IA (similarité groupes + résumés)
- Dashboard personnalisable
- Export CSV/PDF/infographies
- Pipeline CI/CD automatisé

WON'T

- Streaming temps réel
- Infrastructure cloud complexe
- Application mobile native
- Fonctionnalités sociales avancées

Plan de Sprints – 5 itérations



DEEP DIVE

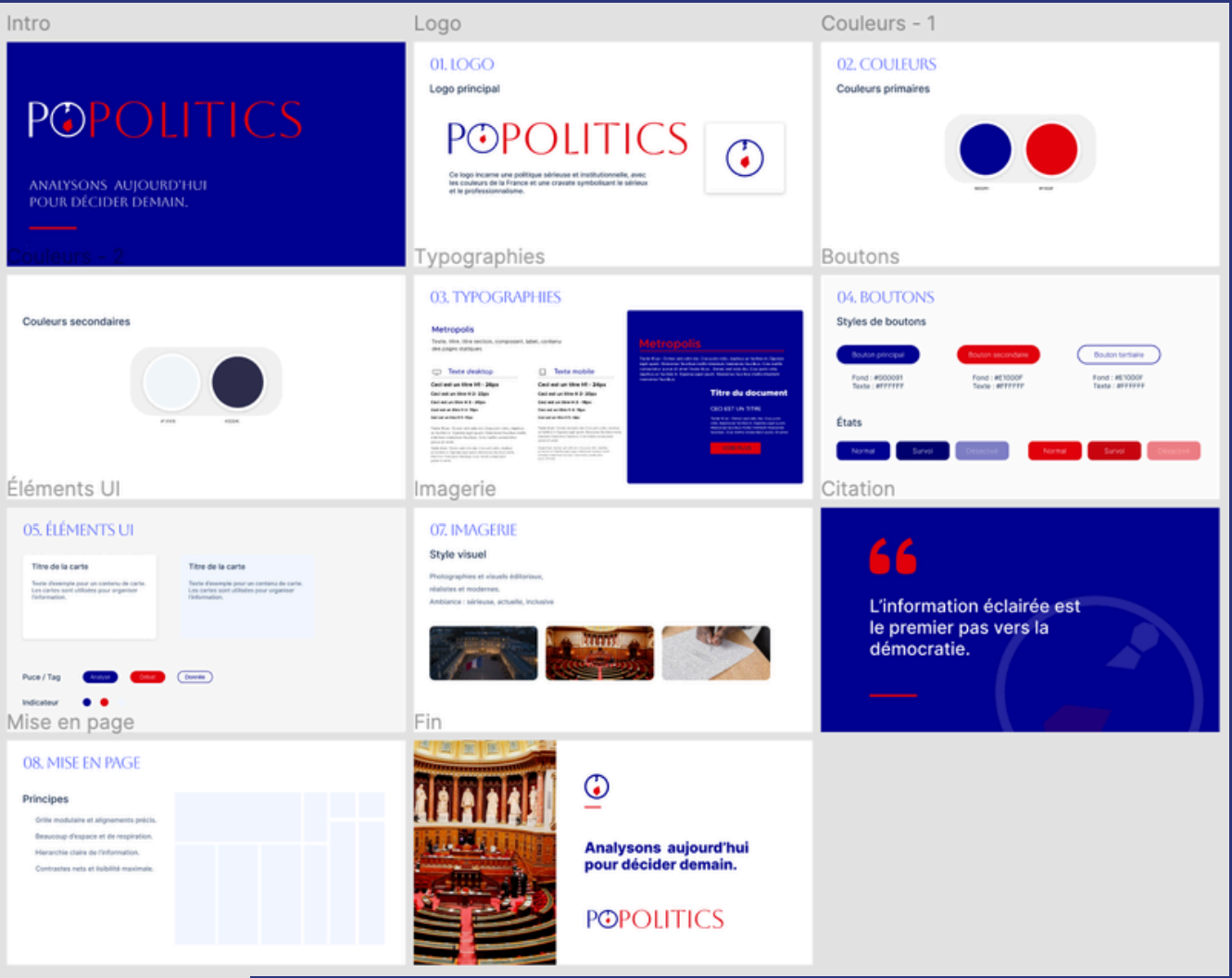
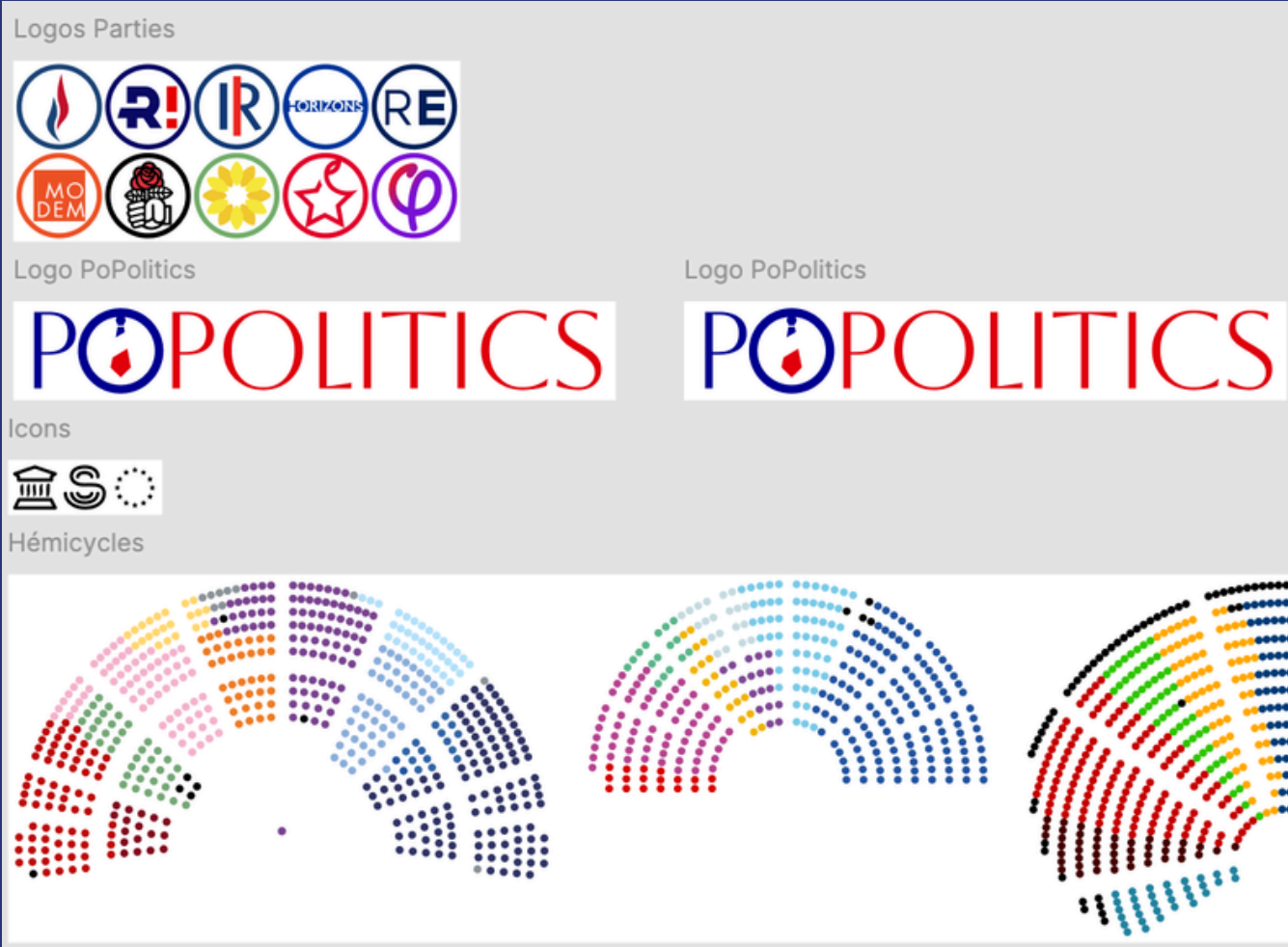
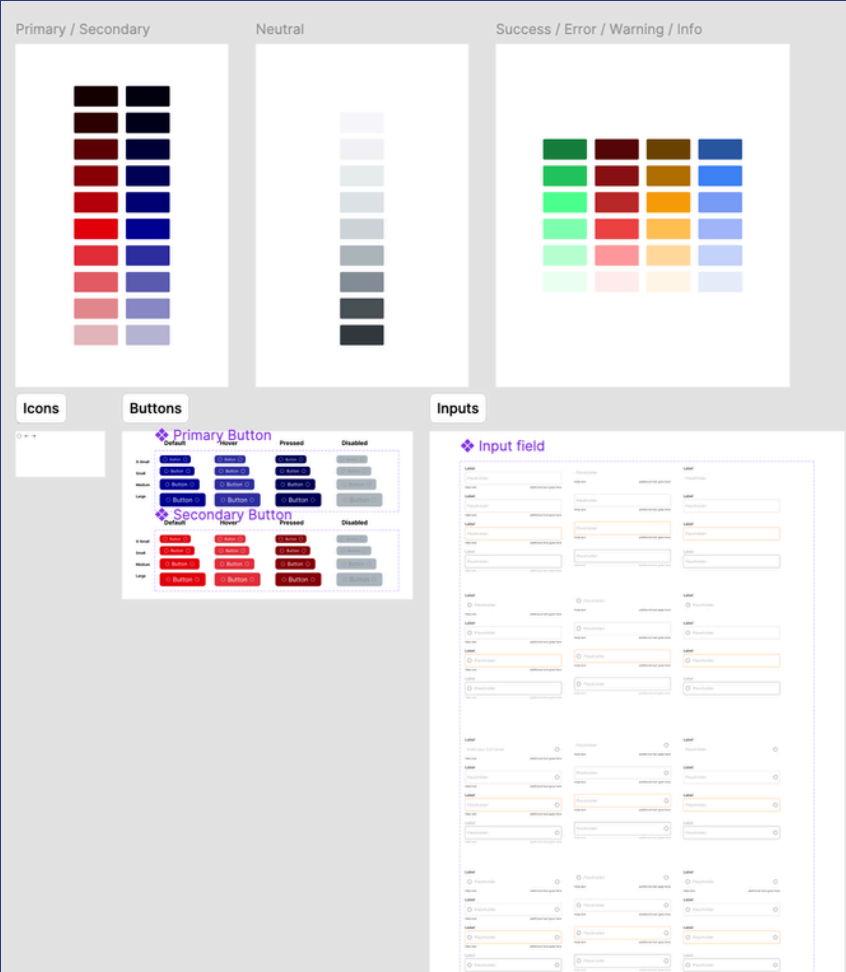
Technique

01 Outils répondant au besoin

02 Architecture de l'outil

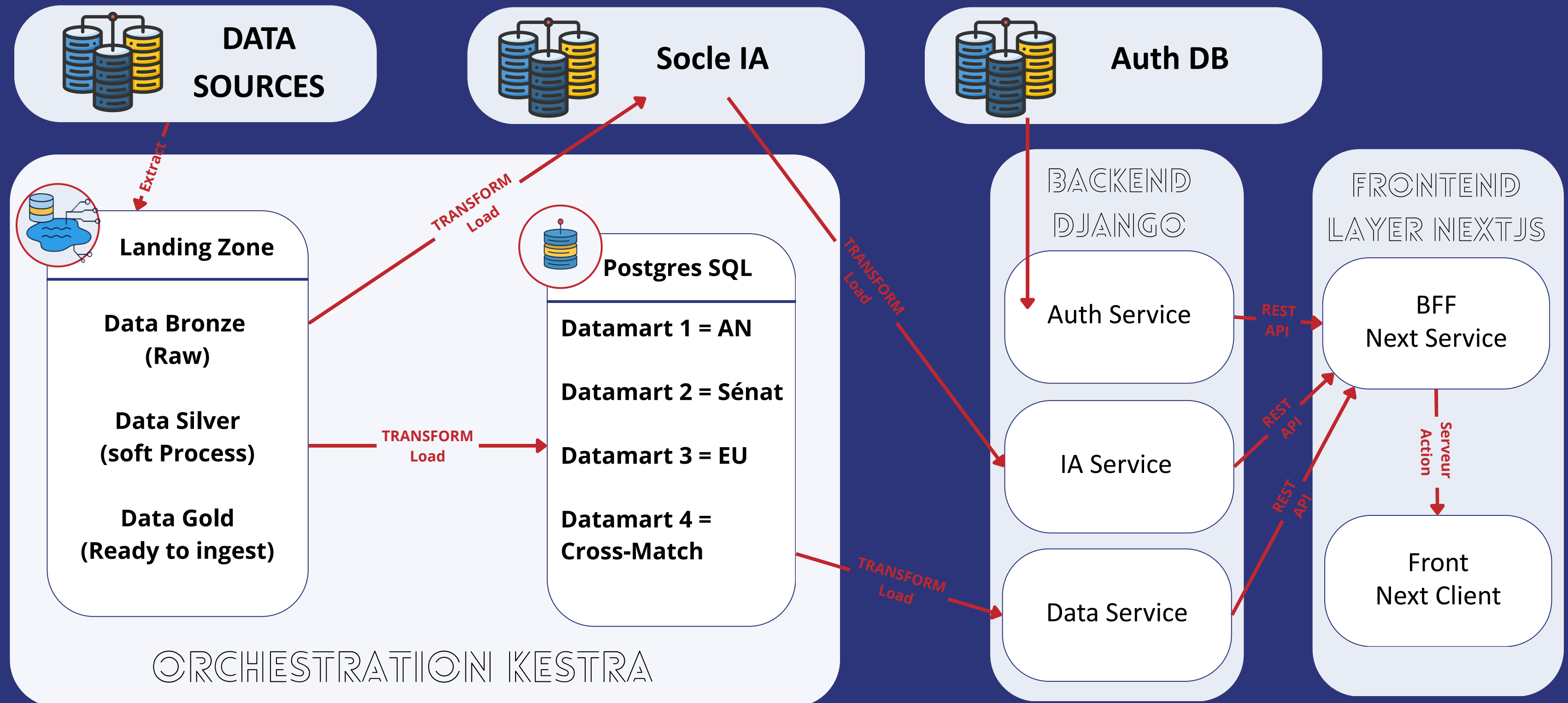
03 Scalabilité et coûts

Outils répondant au besoin



L-8 À découper

Architecture



Architecture



Socle IA

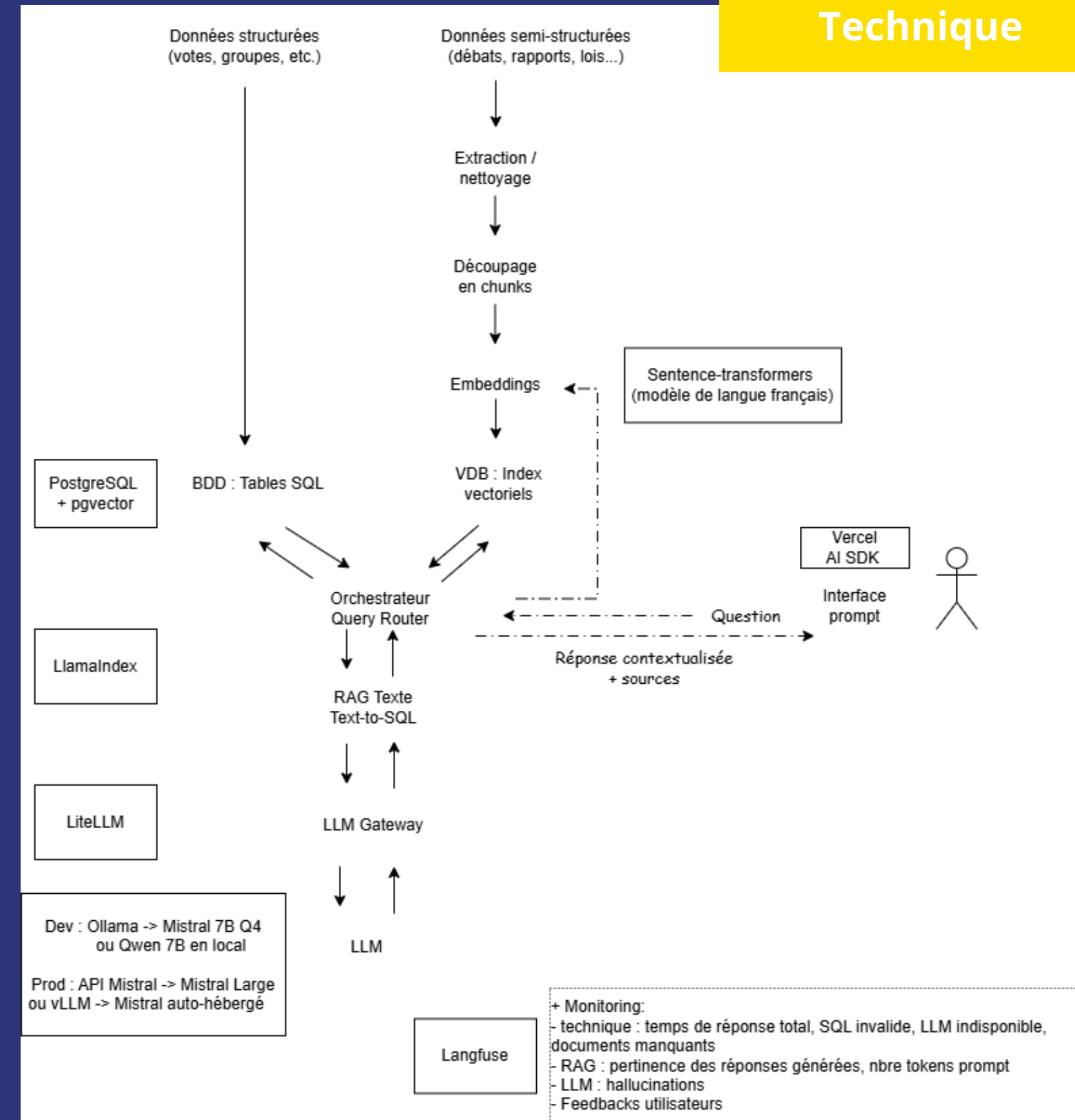
Traitement des données textuelles

Similarité sémantique - Déclaration des groupes
Résumés automatiques des lois / amendements

Chatbot

Analyse des tendances dans les discours et
prises de parole

Technique



Scalabilité et coûts

MVP Local

Docker sur laptop dev

0 €

Réutilisation matériel existant

Configuration cible :

4 vCPU · 16 Go RAM · 256 Go SSD

V1 — Serveur

On-premise léger

0–1 200 €

Réutilisation ou achat one-shot

Configuration cible :

4–8 vCPU · 16–32 Go RAM · 512 Go–1 To

Cloud

VM managée (OVH/GCP/Azure)

45–100 €/mois

VM + stockage (estimate)

Configuration cible :

4 vCPU · 16 Go RAM · 500 Go SSD

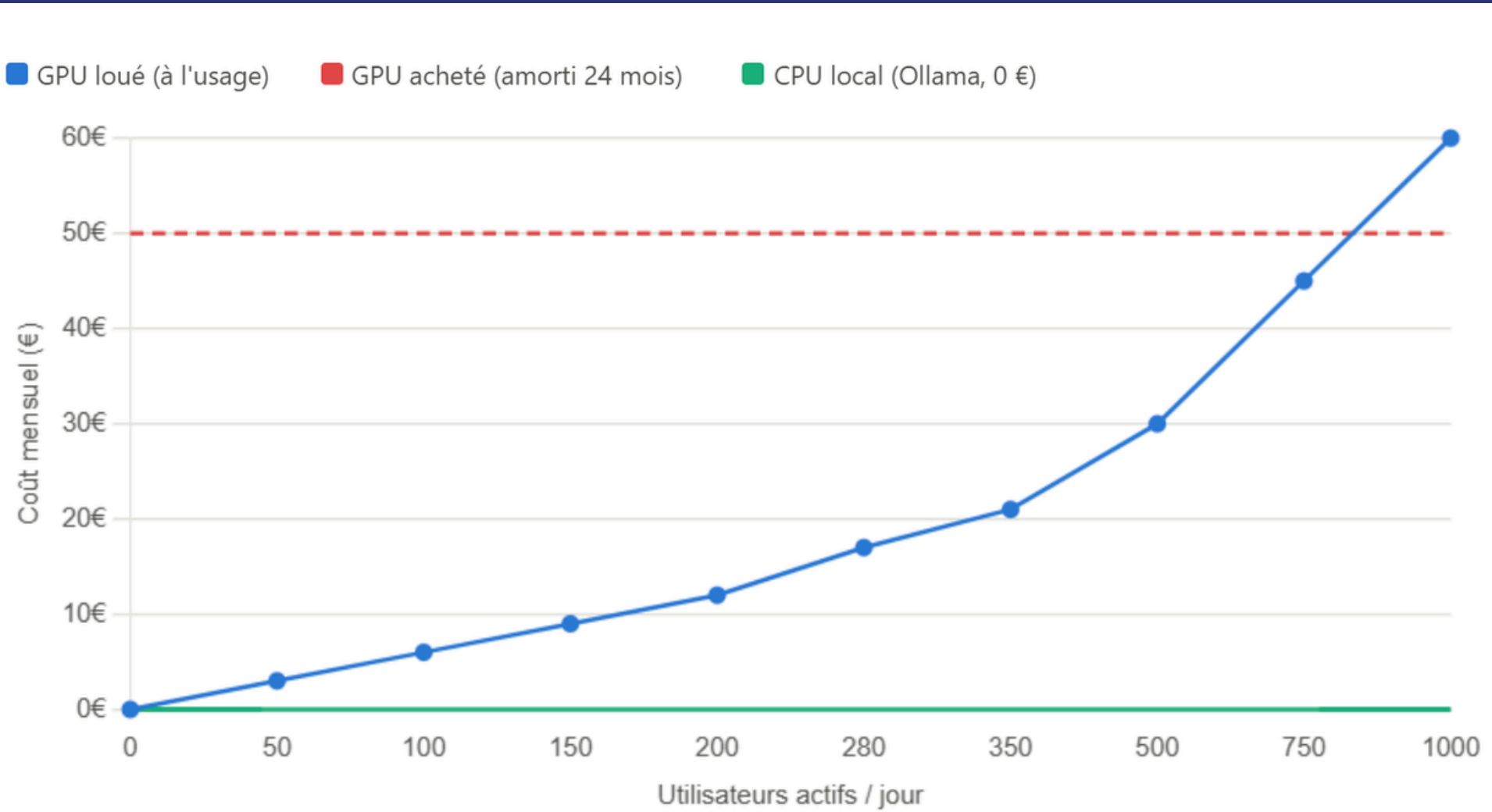
Scalabilité et coûts Chatbot IA

- POC / MVP : inférence locale avec Ollama/Mistral pour limiter les coûts et garantir la souveraineté des données.
- Montée en charge : évolution progressive vers une infrastructure GPU en fonction du nombre d'utilisateurs
- Optimisation des coûts : chaque investissement est déclenché uniquement lorsqu'il devient plus rentable que la solution actuelle.

Volume	Coût API (Mistral Small)	Coût GPU loué
200 utilisateurs × 4 req/jour	3-4 €/mois	12 €/mois
1 000 utilisateurs × 4 req/jour	18 €/mois	60 €/mois
5 000 utilisateurs × 4 req/jour	90 €/mois	300 €/mois

Hypothèses : Mistral Small à 0,10 \$/M tokens entrée, 0,30 \$/M sortie ; ~600 tokens entrée + 300 sortie par requête. GPU loué à 0,45 €/h, 4s de calcul par requête.

Stratégie d'évolution de l'infrastructure IA



Hypothèses : 4 requêtes/utilisateur/jour, 4s de calcul GPU/requête, location à 0,45 €/h, achat 1 200 € amorti sur 24 mois.

Solutions de financement

Crowdfunding

& Dons de particuliers
anonymisés



Appels à projet publics

Accélérateur d'initiatives citoyennes
- DINUM



Sponsors privés



Mistral AI

Appels à projets associatifs et communautés
tech



**DATA FOR
GOOD**

POPolitics - Stack Technique

ETL	Kestra · Data Lake Bronze/Silver/Gold · Parquet	 
Data	4 Datamarts PostgreSQL · Cross-match multi-institutions	 
IA	NLP · Résumés / Chatbot · Analyse des tendances	 
Backend	Django Python · REST APIs · 3 services (Data/IA/Auth)	 
Frontend	Next.js · BFF · SSR/SSG · Design System unifié	 
Qualité	DoD stricte · Code review · CI · Tests unitaires/intégration	 
Dev Practices	Conventional Commits · Git Flow · PR obligatoire x2	

Kestra vs Airflow — Pourquoi Kestra ?



KESTRA ✓ Choix retenu

- Déclaratif YAML — lisible par toute l'équipe
- Interface moderne intégrée (logs, monitoring, retries)
- Déploiement Docker simple (1 conteneur)
- Orienté data engineering nativement
- Outil français — cohérence éthique du projet
- Courbe d'apprentissage adaptée à une équipe étudiante
- Communauté plus jeune qu'Airflow



AIRFLOW — Non retenu (MVP)

- Standard industriel très répandu
- Très puissant, riche en opérateurs
- Communauté large, documentation exhaustive
- Lourd à installer et administrer
- Courbe d'apprentissage importante (Python DAGs)
- Configuration complexe pour un projet étudiant
- Envisageable post-MVP si scaling nécessaire

VS

Django vs FastAPI vs Node.js — Pourquoi Django ?

Django / Python

✓ RETENU

✓ Avantages

- ORM intégré → accès facile aux datamarts PostgreSQL
- Module Auth intégré (Auth Service clé en main)
- Cohérence totale avec l'écosystème Python (ETL + IA)
- Structuration multi-app naturelle (Data/IA/Auth Services)
- Admin auto-généré pour debug/supervision

✗ Limites

- Plus verbeux que FastAPI
- Un peu plus lourd pour des micro-services légers

FastAPI / Python

✓ Avantages

- Très rapide, typing moderne
- Parfait pour micro-services légers
- Documentation OpenAPI automatique

✗ Limites

- Doit assembler soi-même auth, ORM, admin...
- Moins adapté à un backend complet « tout-en-un »
- Réservé éventuellement au socle IA (FastAPI comme IA service)

Node.js / Express

✓ Avantages

- Homogénéité full-JS avec le frontend Next.js
- Grande communauté, riche écosystème NPM

✗ Limites

- Pont JS ↔ Python permanent pour ETL et IA
- Moins naturel pour interfacer les pipelines Python
- Complexité de maintenance entre deux mondes

Next.js vs React SPA — Pourquoi Next.js ?

Next.js ✓ Choix retenu

- SSR / SSG natif → performance & SEO optimisés
 - BFF intégré (Next Server + Server Actions)
 - Logique sensible restée côté serveur → sécurité
 - Intégration naturelle avec APIs REST Django
 - Déploiement flexible (Vercel, Docker, self-hosted)
-
- Architecture plus avancée à appréhender
 - Build plus long qu'une SPA simple

VS

React SPA classique — Non retenu

- Simplicité de mise en place
 - Très répandu, bien connu des étudiants
 - Communauté et écosystème énorme
-
- Performances moindres
 - Logique sensible exposée côté client
 - Nécessite un proxy/BFF séparé pour sécurité API

Marketing et communication

Rendre l'activité parlementaire lisible, crédible et engageante pour le grand public

Marketing

Problème : données politiques disponibles mais complexes à comprendre.

Positionnement : plateforme civic-tech non partisane, centrée sur la transparence.

Cibles : citoyens engagés, journalistes/data journalistes, experts.

Proposition de valeur : données officielles + expérience simple + personnalisation.

Différenciation : croisement Assemblée nationale / Sénat / Parlement européen.

Communication externe

Objectifs : notoriété, crédibilité, engagement, légitimité académique.

Canaux : GitHub, LinkedIn, X.

Ligne éditoriale : factuelle, accessible, apolitique, rigoureuse.

Rythme : communication calée sur le calendrier éditorial.

Posture : Professionnel, engagé, pédagogique

Conclusion



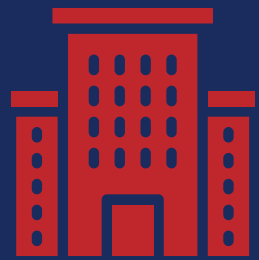
Transparence



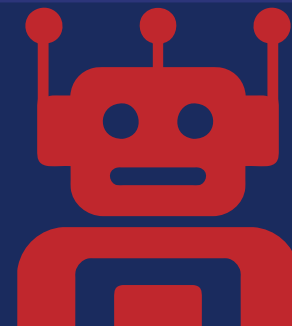
Lisibilité



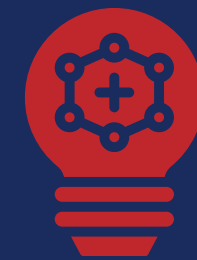
Confiance



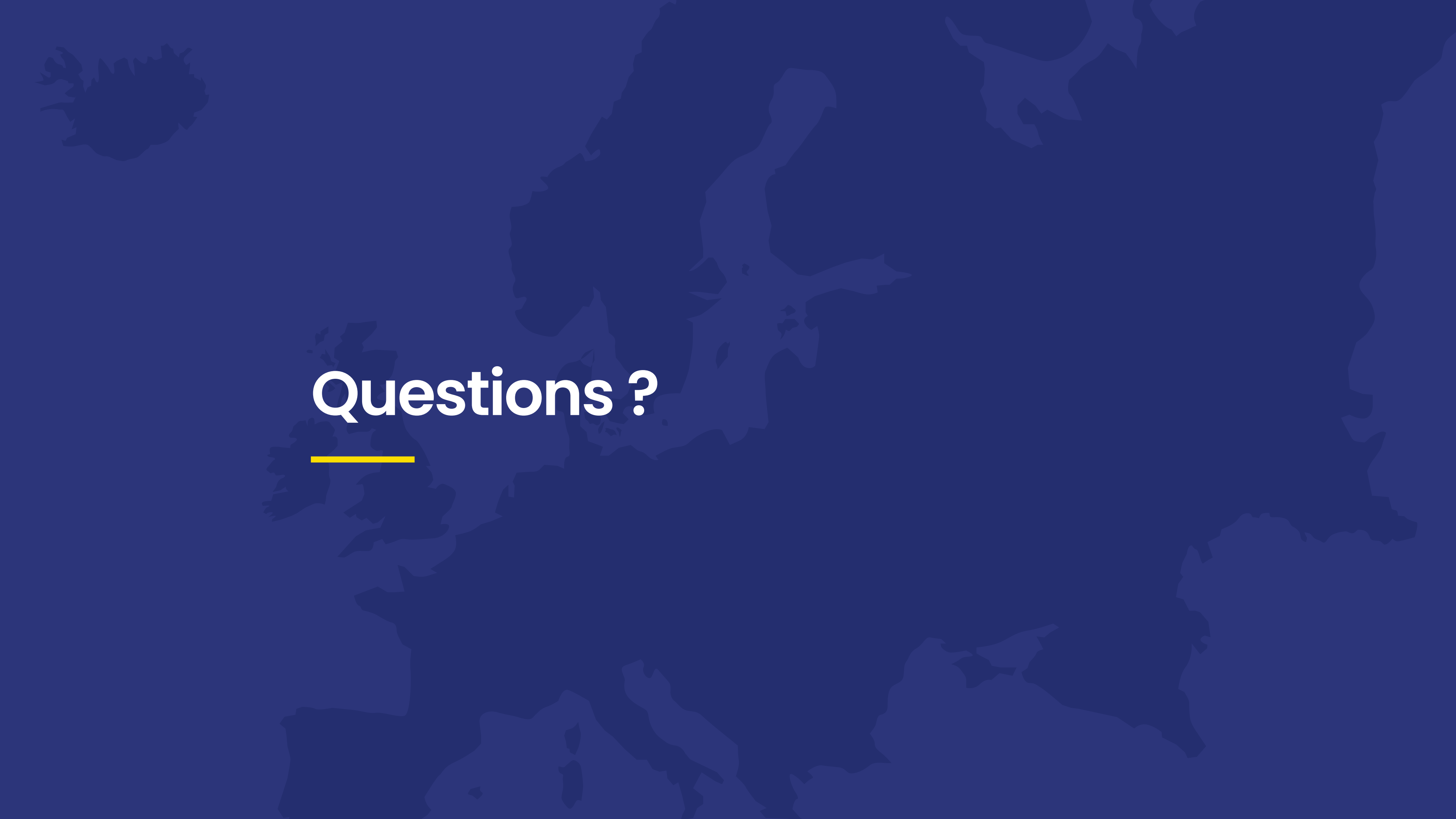
Architecture



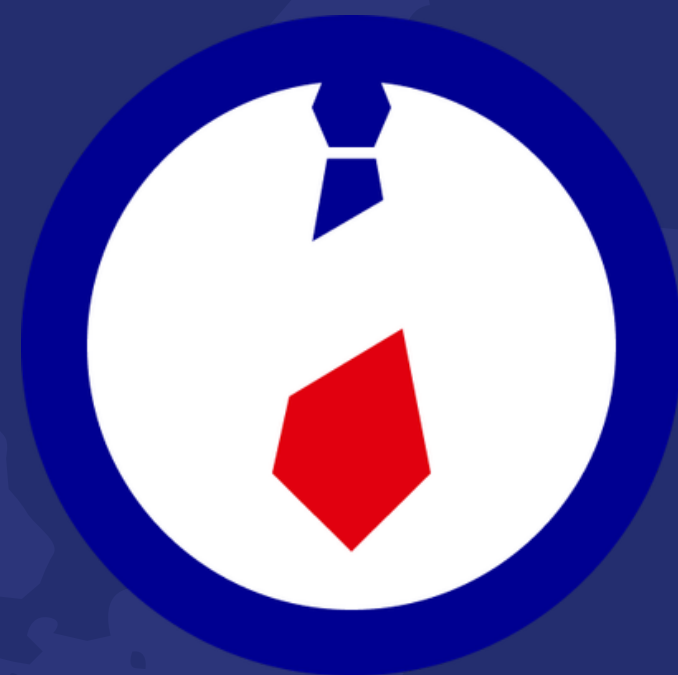
Synthèses IA



Organisation



Questions ?



Merci pour votre attention !

